NTNU

Inference in First-Order Logic

Ole C. Eidheim

October 1, 2026

Department of Computer Science

Outline

Introduction

Propositionalization

Generalized Modus Ponens

Resolution

Checking for entailment in first-order logic is generally semidecidable

Only entailed sentences can be proven with certainty in finite time

---

Checking for entailment is **decidable** for knowledge bases consisting of only definite clauses and no functions

Checking for entailment in first-order logic is generally semidecidable

Only entailed sentences can be proven with certainty in finite time

Review the part of the Peano axioms that define an infinite sequence of distinct objects:

$$KB = \{NatNum(0),$$
$$\forall x\ NatNum(x) \Rightarrow NatNum(Successor(x)),$$
$$\forall x\ \neg(Successor(x) = 0),$$
$$\forall x, y\ \neg(x = y) \Rightarrow \neg(Successor(x) = Successor(y))\}$$

Checking for entailment is **decidable** for knowledge bases consisting of only definite clauses and no functions

Checking for entailment in first-order logic is generally semidecidable

Only entailed sentences can be proven with certainty in finite time

Review the part of the Peano axioms that define an infinite sequence of distinct objects:

$$KB = \{NatNum(0),$$
$$\forall x\ NatNum(x) \Rightarrow NatNum(Successor(x)),$$
$$\forall x\ \neg(Successor(x) = 0),$$
$$\forall x, y\ \neg(x = y) \Rightarrow \neg(Successor(x) = Successor(y))\}$$

Proving $KB \models NatNum(Successor(Successor(0)))$ can be done in finite time, if entailment checks are performed in between function assignments of increasing depth, e.g.: check, $Successor = \{0 = 1\}$, check, $Successor = \{0 = 1, 1 = 2\}$, check, ...

Checking for entailment is **decidable** for knowledge bases consisting of only definite clauses and no functions

Checking for entailment in first-order logic is generally semidecidable

Only entailed sentences can be proven with certainty in finite time

Review the part of the Peano axioms that define an infinite sequence of distinct objects:

$$KB = \{NatNum(0),$$
$$\forall x\ NatNum(x) \Rightarrow NatNum(Successor(x)),$$
$$\forall x\ \neg(Successor(x) = 0),$$
$$\forall x, y\ \neg(x = y) \Rightarrow \neg(Successor(x) = Successor(y))\}$$

Proving $KB \models NatNum(Successor(Successor(0)))$ can be done in finite time, if entailment checks are performed in between function assignments of increasing depth, e.g.: check, $Successor = \{0 = 1\}$, check, $Successor = \{0 = 1, 1 = 2\}$, check, ...

However, proving $KB \not\models \neg NatNum(Successor(Successor(0)))$ cannot be done using proof by contradiction, since $KB \cup \{\neg\neg NatNum(Successor(Successor(0)))\}$ contains no contradictions and all the models in $M(KB)$ are infinite

Checking for entailment is **decidable** for knowledge bases consisting of only definite clauses and no functions

Outline

Introduction

Propositionalization

Generalized Modus Ponens

Resolution

Propositionalization example

Convert first-order sentences to propositional sentences, e.g. to perform propositional inference:

$$KB = \{Student(Alice),$$
$$Student(Bob),$$
$$\forall x\ Student(x) \Rightarrow Knows(x, Arithmetic)\}$$

Propositionalization example

Convert first-order sentences to propositional sentences, e.g. to perform propositional inference:

1. Replace all variables with constants or functions

$$
\begin{aligned}
KB = \{&Student(Alice),\\
&Student(Bob),\\
&\forall x\ Student(x) \Rightarrow Knows(x, Arithmetic),\\
&Student(Alice) \Rightarrow Knows(Alice, Arithmetic),\\
&Student(Bob) \Rightarrow Knows(Bob, Arithmetic),\\
&Student(Arithmetic) \Rightarrow Knows(Arithmetic, Arithmetic)\}
\end{aligned}
$$

Propositionalization example

Convert first-order sentences to propositional sentences, e.g. to perform propositional inference:

1. Replace all variables with constants or functions
2. Replace remaining atomic sentences with propositional symbols

$$
\begin{aligned}
KB = \{ & \mathit{Student}(\mathit{Alice}), \\
& \mathit{Student}(\mathit{Bob}), \\
& \forall x\ \mathit{Student}(x) \Rightarrow \mathit{Knows}(x, \mathit{Arithmetic}), \\
& \mathit{Student}(\mathit{Alice}) \Rightarrow \mathit{Knows}(\mathit{Alice}, \mathit{Arithmetic}), \\
& \mathit{Student}(\mathit{Bob}) \Rightarrow \mathit{Knows}(\mathit{Bob}, \mathit{Arithmetic}), \\
& \mathit{Student}(\mathit{Arithmetic}) \Rightarrow \mathit{Knows}(\mathit{Arithmetic}, \mathit{Arithmetic}), \\
& \mathit{AliceIsStudent}, \\
& \mathit{BobIsStudent}, \\
& \mathit{AliceIsStudent} \Rightarrow \mathit{AliceKnowsArithmetic}, \\
& \mathit{BobIsStudent} \Rightarrow \mathit{BobKnowsArithmetic}, \\
& \mathit{ArithmeticIsStudent} \Rightarrow \mathit{ArithmeticKnowsArithmetic} \quad \}
\end{aligned}
$$

Substitution

Substitution

Maps variables to terms (constants, variables or functions)

$\textsc{Subst}(\theta, \alpha)$: returns the result of applying substitution θ to sentence α

Examples:

$$\textsc{Subst}(\{x/\mathit{Alice}\}, \mathit{Knows}(x, \mathit{Arithmetic})) = \mathit{Knows}(\mathit{Alice}, \mathit{Arithmetic})$$

$$\textsc{Subst}(\{x/y\}, \mathit{Knows}(x, \mathit{Arithmetic})) = \mathit{Knows}(y, \mathit{Arithmetic})$$

$$\textsc{Subst}(\{x/\mathit{Mother}(\mathit{Alice})\}, \mathit{Knows}(x, \mathit{Arithmetic})) = \mathit{Knows}(\mathit{Mother}(\mathit{Alice}), \mathit{Arithmetic})$$

Universal instantiation inference rule

Universal instantiation inference rule

For any variable v, sentence α, and ground term g (constant or function):

$$\frac{\forall v\ \alpha}{\textsc{Subst}(\{v/g\}, \alpha)}$$

For example:

$$\frac{\forall x\ \mathit{Student}(x) \Rightarrow \mathit{Knows}(x, \mathit{English})}{\textsc{Subst}(\{x/\mathit{Alice}\}, \mathit{Student}(x) \Rightarrow \mathit{Knows}(x, \mathit{English}))}$$

results in:

$$\frac{\forall x\ \mathit{Student}(x) \Rightarrow \mathit{Knows}(x, \mathit{English})}{\mathit{Student}(\mathit{Alice}) \Rightarrow \mathit{Knows}(\mathit{Alice}, \mathit{English})}$$

Universal instantiation inference rule

Universal instantiation inference rule

For any variable v, sentence α, and ground term g (constant or function):

$$\frac{\forall v\ \alpha}{\textsc{Subst}(\{v/g\}, \alpha)}$$

Example use of the universal instantiation inference rule:

$KB = \{Student(Alice), \forall x\ Student(x) \Rightarrow Knows(x, English)\}$

Universal instantiation inference rule

Universal instantiation inference rule

For any variable v, sentence α, and ground term g (constant or function):

$$\frac{\forall v\ \alpha}{\textsc{Subst}(\{v/g\}, \alpha)}$$

Example use of the universal instantiation inference rule:

$KB = \{Student(Alice), \forall x\ Student(x) \Rightarrow Knows(x, English)\}$

Apply universal instantiation to $\forall x$ and $Student(x) \Rightarrow Knows(x, English)$ with substitution $\{x/Alice\}$ to derive $Student(Alice) \Rightarrow Knows(Alice, English)$:

$KB = \{Student(Alice), \forall x\ Student(x) \Rightarrow Knows(x, English),$
$Student(Alice) \Rightarrow Knows(Alice, English) \}$

Universal instantiation inference rule

Universal instantiation inference rule

For any variable v, sentence α, and ground term g (constant or function):

$$\frac{\forall v\ \alpha}{\textsc{Subst}(\{v/g\}, \alpha)}$$

Example use of the universal instantiation inference rule:

$KB = \{Student(Alice), \forall x\ Student(x) \Rightarrow Knows(x, English)\}$

Apply universal instantiation to $\forall x$ and $Student(x) \Rightarrow Knows(x, English)$ with substitution $\{x/Alice\}$ to derive $Student(Alice) \Rightarrow Knows(Alice, English)$:

$KB = \{Student(Alice), \forall x\ Student(x) \Rightarrow Knows(x, English),$
$Student(Alice) \Rightarrow Knows(Alice, English) \}$

Apply universal instantiation to $\forall x$ and $Student(x) \Rightarrow Knows(x, English)$ with substitution $\{x/English\}$ to derive $Student(English) \Rightarrow Knows(English, English)$:

$KB = \{Student(Alice), \forall x\ Student(x) \Rightarrow Knows(x, English),$
$Student(Alice) \Rightarrow Knows(Alice, English),\ Student(English) \Rightarrow Knows(English, English)\}$

Existential instantiation inference rule

Existential instantiation inference rule

For any variable v, sentence α, and a Skolem constant k not already in the KB:

$$\frac{\exists v\ \alpha}{\textsc{Subst}(\{v/k\}, \alpha)}$$

For example:

$$\frac{\exists x\ \mathit{Student}(x) \land \mathit{Knows}(x, \mathit{Norwegian})}{\textsc{Subst}(\{x/C_1\}, \mathit{Student}(x) \land \mathit{Knows}(x, \mathit{Norwegian}))}$$

results in:

$$\frac{\exists x\ \mathit{Student}(x) \land \mathit{Knows}(x, \mathit{Norwegian})}{\mathit{Student}(C_1) \land \mathit{Knows}(C_1, \mathit{Norwegian})}$$

Existential instantiation inference rule

Existential instantiation inference rule

For any variable v, sentence α, and a Skolem constant k not already in the KB:

$$\frac{\exists v\ \alpha}{\textsc{Subst}(\{v/k\}, \alpha)}$$

For example:

$$\frac{\exists x\ \mathit{Student}(x) \land \mathit{Knows}(x, \mathit{Norwegian})}{\textsc{Subst}(\{x/C_1\}, \mathit{Student}(x) \land \mathit{Knows}(x, \mathit{Norwegian}))}$$

results in:

$$\frac{\exists x\ \mathit{Student}(x) \land \mathit{Knows}(x, \mathit{Norwegian})}{\mathit{Student}(C_1) \land \mathit{Knows}(C_1, \mathit{Norwegian})}$$

Intuition: $\exists x\ \mathit{Student}(x) \land \mathit{Knows}(x, \mathit{Norwegian})$ is *true*, therefore there must exist an object, that we name C_1, for which $\mathit{Student}(C_1) \land \mathit{Knows}(C_1, \mathit{Norwegian})$ is *true*

Existential instantiation inference rule

Existential instantiation inference rule

For any variable v, sentence α, and a Skolem constant k not already in the KB:

$$\frac{\exists v\ \alpha}{\textsc{Subst}(\{v/k\}, \alpha)}$$

Example use of the existential instantiation inference rule:

$KB = \{Student(Alice), \exists x\ Student(x) \wedge Knows(x, Norwegian)\}$

Existential instantiation inference rule

Existential instantiation inference rule

For any variable v, sentence α, and a Skolem constant k not already in the KB:

$$\frac{\exists v\ \alpha}{\textsc{Subst}(\{v/k\}, \alpha)}$$

Example use of the existential instantiation inference rule:

$KB = \{Student(Alice), \exists x\ Student(x) \wedge Knows(x, Norwegian)\}$

Apply existential instantiation to $\exists x$ and $Student(x) \wedge Knows(x, Norwegian)$ with substitution $\{x/C_1\}$ to derive $Student(C_1) \wedge Knows(C_1, Norwegian)$:

$KB = \{Student(Alice), \exists x\ Student(x) \wedge Knows(x, Norwegian),$
$Student(C_1) \wedge Knows(C_1, Norwegian)\ \}$

Existential instantiation inference rule

Existential instantiation inference rule

For any variable v, sentence α, and a Skolem constant k not already in the KB:

$$\frac{\exists v\ \alpha}{\textsc{Subst}(\{v/k\}, \alpha)}$$

Note that if the KB already include sentences that make the existential sentence *true*, we can disregard it without introducing a Skolem constant, e.g.:

$$KB = \{Student(Alice), Knows(Alice, Norwegian),$$
$$\exists x\ Student(x) \wedge Knows(x, Norwegian) \quad \}$$

For example, the Z3 Theorem Prover does not introduce Skolem constants in such cases

Existential instantiation, Skolem functions

Skolem constants are sometimes insufficient, for example:

$$\forall x\ \exists y\ \mathit{Loves}(x, y) \qquad \text{Everyone loves someone}$$

is not the same as:

$$\forall x\ \mathit{Loves}(x, C_1) \qquad \text{Everyone loves } C_1$$

Existential instantiation, Skolem functions

Skolem constants are sometimes insufficient, for example:

$$\forall x\ \exists y\ \mathit{Loves}(x, y) \qquad \text{Everyone loves someone}$$

is not the same as:

$$\forall x\ \mathit{Loves}(x, C_1) \qquad \text{Everyone loves } C_1$$

In such cases, we need to use Skolem functions instead:

$$\forall x\ \exists y\ \mathit{Loves}(x, y) \qquad \text{Everyone loves someone}$$

is the same as:

$$\forall x\ \mathit{Loves}(x, F(x)) \qquad \text{Everyone loves someone}$$

Propositionalization, disadvantage

Propositionalization may generate a significant number of unnecessary derivations from universally quantified sentences, e.g.:

Given $KB = \{Student(Alice),$
$Student(Bob),$
$\forall x\ Student(x) \Rightarrow Knows(x, Arithmetic)\}$

deciding $KB \models Knows(Alice, Arithmetic)$ does not require the derivation of $Student(Arithmetic) \Rightarrow Knows(Arithmetic, Arithmetic)$, but propositionalization blindly adds this derivation to the KB

Propositionalization, disadvantage

Propositionalization may generate a significant number of unnecessary derivations from universally quantified sentences, e.g.:

Given $KB = \{Student(Alice),$
$Student(Bob),$
$\forall x\ Student(x) \Rightarrow Knows(x, Arithmetic)\}$

deciding $KB \models Knows(Alice, Arithmetic)$ does not require the derivation of $Student(Arithmetic) \Rightarrow Knows(Arithmetic, Arithmetic)$, but propositionalization blindly adds this derivation to the KB

However, propositional logic inference algorithms can be extended to first-order logic

Outline

Introduction

Propositionalization

Generalized Modus Ponens

Resolution

Unification

Review: substitution

Maps variables to terms (constants, variables or functions)

$\textsc{Subst}(\theta, \alpha)$: returns the result of performing substitution θ to sentence α

Figure 9.1 in the book shows a unification algorithm

Unification

Review: substitution

Maps variables to terms (constants, variables or functions)

$\textsc{Subst}(\theta, \alpha)$: returns the result of performing substitution θ to sentence α

Unification

For any sentence α and β, $\textsc{Unify}(\alpha, \beta)$ attempts to find the most general substitution θ that makes the substituted sentences identical:

$$\textsc{Unify}(\alpha, \beta) = \theta \text{ such that } \textsc{Subst}(\theta, \alpha) = \textsc{Subst}(\theta, \beta)$$

Figure 9.1 in the book shows a unification algorithm

Unification

Review: substitution

Maps variables to terms (constants, variables or functions)

$\textsc{Subst}(\theta, \alpha)$: returns the result of performing substitution θ to sentence α

Unification

For any sentence α and β, $\textsc{Unify}(\alpha, \beta)$ attempts to find the most general substitution θ that makes the substituted sentences identical:

$$\textsc{Unify}(\alpha, \beta) = \theta \text{ such that } \textsc{Subst}(\theta, \alpha) = \textsc{Subst}(\theta, \beta)$$

Examples:

$$\textsc{Unify}(\mathit{Loves}(x, \mathit{Arithmetic}), \mathit{Loves}(\mathit{Alice}, \mathit{Arithmetic})) = \{x/\mathit{Alice}\}$$

$$\textsc{Unify}(\mathit{Loves}(x, y), \mathit{Loves}(\mathit{Alice}, z)) = \{x/\mathit{Alice}, y/z\}$$

$$\textsc{Unify}(\mathit{Loves}(\mathit{Bob}, y), \mathit{Loves}(\mathit{Alice}, z)) = \mathit{failure}$$

$$\textsc{Unify}(\mathit{Loves}(\mathit{Bob}, y), \mathit{Loves}(x, \mathit{Mother}(x))) = \{x/\mathit{Bob}, y/\mathit{Mother}(\mathit{Bob})\}$$

Figure 9.1 in the book shows a unification algorithm

Unification, time complexity

There exists unification algorithms with linear time complexity in the size of the expressions being unified

Generalized Modus Ponens, without quantifiers

Review: Modus Ponens

For any sentence α, β, if α is *true* and $\alpha \Rightarrow \beta$ is *true*, β must also be *true*:

$$\frac{\alpha, \quad \alpha \Rightarrow \beta}{\beta}$$

Generalized Modus Ponens (GMP) without quantifiers

For any atomic sentences $p_1, p_2, \ldots, p_n, q$:

$$\frac{p_1, p_2, \ldots, p_n, \qquad p_1 \wedge p_2 \wedge \ldots \wedge p_n \Rightarrow q}{q}$$

Generalized Modus Ponens, without quantifiers

Generalized Modus Ponens (GMP) without quantifiers

For any atomic sentences $p_1, p_2, \ldots, p_n, q$:

$$\frac{p_1, p_2, \ldots, p_n, \qquad p_1 \wedge p_2 \wedge \ldots \wedge p_n \Rightarrow q}{q}$$

For example, given

$$KB = \{Takes(Alice, TDT4136), Covers(TDT4136, Logic),$$
$$Takes(Alice, TDT4136) \wedge Covers(TDT4136, Logic) \Rightarrow Knows(Alice, Logic)\}$$

Does $KB \models Knows(Alice, Logic)$?

Generalized Modus Ponens, without quantifiers

Generalized Modus Ponens (GMP) without quantifiers

For any atomic sentences $p_1, p_2, \ldots, p_n, q$:

$$\frac{p_1, p_2, \ldots, p_n, \qquad p_1 \wedge p_2 \wedge \ldots \wedge p_n \Rightarrow q}{q}$$

For example, given

$KB = \{Takes(Alice, TDT4136), Covers(TDT4136, Logic),$
$Takes(Alice, TDT4136) \wedge Covers(TDT4136, Logic) \Rightarrow Knows(Alice, Logic)\}$

Does $KB \models Knows(Alice, Logic)$?

Apply Generalized Modus Ponens to $Takes(Alice, TDT4136)$, $Covers(TDT4136, Logic)$ and $Takes(Alice, TDT4136) \wedge Covers(TDT4136, Logic) \Rightarrow Knows(Alice, Logic)$ to derive $Knows(Alice, Logic)$

Answer: yes, $KB \vdash_{GMPA} Knows(Alice, Logic)$, therefore $KB \models Knows(Alice, Logic)$

Generalized Modus Ponens

Generalized Modus Ponens (GMP)

For any atomic sentences $p'_1, p'_2, \ldots, p'_n, p_1, p_2, \ldots, p_n, q$ with variables $x_1, x_2, \ldots$:

$$\frac{p'_1, p'_2, \ldots, p'_n, \qquad \forall x_1, x_2, \ldots \;\; p_1 \wedge p_2 \wedge \ldots \wedge p_n \Rightarrow q}{\textsc{Subst}(\theta, q)}$$

where $\theta = \textsc{Unify}(p'_1 \wedge p'_2 \wedge \ldots \wedge p'_n, p_1 \wedge p_2 \wedge \ldots \wedge p_n)$

Generalized Modus Ponens

Generalized Modus Ponens (GMP)

For any atomic sentences $p'_1, p'_2, \ldots, p'_n, p_1, p_2, \ldots, p_n, q$ with variables $x_1, x_2, \ldots$:

$$\frac{p'_1, p'_2, \ldots, p'_n, \qquad \forall x_1, x_2, \ldots \;\; p_1 \wedge p_2 \wedge \ldots \wedge p_n \Rightarrow q}{\textsc{Subst}(\theta, q)}$$

where $\theta = \textsc{Unify}(p'_1 \wedge p'_2 \wedge \ldots \wedge p'_n, p_1 \wedge p_2 \wedge \ldots \wedge p_n)$

For example, given $KB = \{ \mathit{Takes}(\mathit{Alice}, \mathit{TDT4136}), \mathit{Covers}(\mathit{TDT4136}, \mathit{Logic}),$
$\forall x, y, z \;\; \mathit{Takes}(x, y) \wedge \mathit{Covers}(y, z) \Rightarrow \mathit{Knows}(x, z) \}$

Does $KB \models \mathit{Knows}(\mathit{Alice}, \mathit{Logic})$?

Generalized Modus Ponens

Generalized Modus Ponens (GMP)

For any atomic sentences $p'_1, p'_2, \ldots, p'_n, p_1, p_2, \ldots, p_n, q$ with variables $x_1, x_2, \ldots$:

$$\frac{p'_1, p'_2, \ldots, p'_n, \qquad \forall x_1, x_2, \ldots \;\; p_1 \wedge p_2 \wedge \ldots \wedge p_n \Rightarrow q}{\textsc{Subst}(\theta, q)}$$

where $\theta = \textsc{Unify}(p'_1 \wedge p'_2 \wedge \ldots \wedge p'_n, p_1 \wedge p_2 \wedge \ldots \wedge p_n)$

For example, given $KB = \{\mathit{Takes}(\mathit{Alice}, \mathit{TDT}4136), \mathit{Covers}(\mathit{TDT}4136, \mathit{Logic}),$
$\forall x, y, z \;\; \mathit{Takes}(x, y) \wedge \mathit{Covers}(y, z) \Rightarrow \mathit{Knows}(x, z)\}$

Does $KB \models \mathit{Knows}(\mathit{Alice}, \mathit{Logic})$?

Apply Generalized Modus Ponens to $\mathit{Takes}(\mathit{Alice}, \mathit{TDT}4136), \mathit{Covers}(\mathit{TDT}4136, \mathit{Logic})$ and $\mathit{Takes}(x, y) \wedge \mathit{Covers}(y, z) \Rightarrow \mathit{Knows}(x, z)$ with substitution $\theta = \{x/\mathit{Alice}, y/\mathit{TDT}4136, z/\mathit{Logic}\}$ to derive $\mathit{Knows}(\mathit{Alice}, \mathit{Logic})$:

Answer: yes, $KB \vdash_{GMPA} \mathit{Knows}(\mathit{Alice}, \mathit{Logic})$, therefore $KB \models \mathit{Knows}(\mathit{Alice}, \mathit{Logic})$

Generalized Modus Ponens, completeness and complexity

Definite clauses in first-order logic

For any atomic sentences $p_1, p_2, \ldots, p_n, q$ with variables $x_1, x_2, \ldots$:

$$\forall x_1, x_2, \ldots \; p_1 \wedge p_2 \wedge \ldots \wedge p_n \Rightarrow q \qquad\qquad q$$

definite clause

fact (definite clause with $n = 0$ and without variables)

For example, $KB = \{ \mathit{Takes}(\mathit{Alice}, \mathit{TDT}4136), \mathit{Covers}(\mathit{TDT}4136, \mathit{Logic}),$

$$\forall x, y, z \;\; \mathit{Takes}(x, y) \wedge \mathit{Covers}(y, z) \Rightarrow \mathit{Knows}(x, z) \}$$

contains only definite clauses

Generalized Modus Ponens, completeness and complexity

Definite clauses in first-order logic

For any atomic sentences $p_1, p_2, \ldots, p_n, q$ with variables $x_1, x_2, \ldots$:

$$\forall x_1, x_2, \ldots \; p_1 \wedge p_2 \wedge \ldots \wedge p_n \Rightarrow q \qquad\qquad q$$

definite clause — fact (definite clause with $n = 0$ and without variables)

For example, $KB = \{ Takes(Alice, TDT4136), Covers(TDT4136, Logic),$
$\forall x, y, z \; Takes(x, y) \wedge Covers(y, z) \Rightarrow Knows(x, z) \}$
contains only definite clauses

If a KB contains only definite clauses, then repeated applications of Generalized Modus Ponens is both sound and complete for any fact q: $KB \models q \Leftrightarrow KB \vdash_{GMPA} q$

Generalized Modus Ponens, completeness and complexity

Definite clauses in first-order logic

For any atomic sentences $p_1, p_2, \ldots, p_n, q$ with variables $x_1, x_2, \ldots$:

$\forall x_1, x_2, \ldots \; p_1 \wedge p_2 \wedge \ldots \wedge p_n \Rightarrow q$ — definite clause

q — fact (definite clause with $n = 0$ and without variables)

For example, $KB = \{ Takes(Alice, TDT4136), Covers(TDT4136, Logic),$
$\forall x, y, z \; Takes(x, y) \wedge Covers(y, z) \Rightarrow Knows(x, z) \}$
contains only definite clauses

If a KB contains only definite clauses, then repeated applications of Generalized Modus Ponens is both sound and complete for any fact q: $KB \models q \Leftrightarrow KB \vdash_{GMPA} q$

If the KB in addition contains no functions, deciding $KB \models q$ can then be done in polynomial time

Outline

Introduction

Propositionalization

Generalized Modus Ponens

Resolution

Literals, clauses and conjunctive normal form in first-order logic

Literal in first-order logic

A first-order literal is an atomic sentence or a negated atomic sentence, e.g. $\neg Loves(x, F(x))$

Literals, clauses and conjunctive normal form in first-order logic

Literal in first-order logic

A first-order literal is an atomic sentence or a negated atomic sentence, e.g.
$\neg Loves(x, F(x))$

Clause in first-order logic

A first-order clause is a disjunction of first-order literals, e.g.
$\neg Loves(x, F(x)) \vee Loves(G(x), x)$

Literals, clauses and conjunctive normal form in first-order logic

Literal in first-order logic

A first-order literal is an atomic sentence or a negated atomic sentence, e.g.
$\neg Loves(x, F(x))$

Clause in first-order logic

A first-order clause is a disjunction of first-order literals, e.g.
$\neg Loves(x, F(x)) \vee Loves(G(x), x)$

Conjunctive normal form in first-order logic

First-order conjunctive normal form is a conjunction of first-order clauses, e.g.
$(Animal(F(x)) \vee Loves(G(x), x)) \wedge (\neg Loves(x, F(x)) \vee Loves(G(x), x))$

Conversion to conjunctive normal form in first-order logic

First-order conjunctive normal form algorithm

1. Eliminate $\Leftrightarrow$: $\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}$
2. Eliminate $\Rightarrow$: $\frac{\alpha \Rightarrow \beta}{\neg \alpha \vee \beta}$
3. Repeatedly move $\neg$ inwards and eliminate double negation: $\frac{\neg(\alpha \wedge \beta)}{\neg\alpha \vee \neg\beta}$ $\frac{\neg(\alpha \vee \beta)}{\neg\alpha \wedge \neg\beta}$ $\frac{\neg(\neg\alpha)}{\alpha}$
4. Standardize variables, e.g. $\frac{\forall x\ (\exists y\ \mathit{Animal}(y) \wedge \neg \mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))}{\forall x\ (\exists y\ \mathit{Animal}(y) \wedge \neg \mathit{Loves}(x,y)) \vee (\exists z\ \mathit{Loves}(z,x))}$
5. Replace existentially quantified variables with Skolem constants or functions
6. Drop universal quantifiers, all variables are now implicitly universally quantified (see reasoning next slide)
7. Distribute $\vee$ over $\wedge$: $\frac{\alpha \vee (\beta \wedge \gamma)}{(\alpha \vee \beta) \wedge (\alpha \vee \gamma)}$ $\frac{(\alpha \wedge \beta) \vee \gamma}{(\alpha \vee \gamma) \wedge (\beta \vee \gamma)}$

Conversion to conjunctive normal form in first-order logic, universal quantifiers

Let KB be:

$$\{\forall x\ Animal(F(x)) \vee Loves(G(x), x), \forall x\ \neg Loves(x, F(x)) \vee Loves(G(x), x)\}$$

KB as sentence:

$$(\forall x\ Animal(F(x)) \vee Loves(G(x), x)) \wedge (\forall x\ \neg Loves(x, F(x)) \vee Loves(G(x), x))$$

Conversion to conjunctive normal form in first-order logic, universal quantifiers

Let KB be:

$$\{\forall x\ Animal(F(x)) \vee Loves(G(x), x), \forall x\ \neg Loves(x, F(x)) \vee Loves(G(x), x)\}$$

KB as sentence:

$$(\forall x\ Animal(F(x)) \vee Loves(G(x), x)) \wedge (\forall x\ \neg Loves(x, F(x)) \vee Loves(G(x), x))$$

Move universal quantifiers outward (see "by hand" proof in last week's lecture):

$$\forall x\ (Animal(F(x)) \vee Loves(G(x), x)) \wedge (\neg Loves(x, F(x)) \vee Loves(G(x), x))$$

Conversion to conjunctive normal form in first-order logic, universal quantifiers

Let KB be:

$$\{\forall x\ Animal(F(x)) \vee Loves(G(x), x), \forall x\ \neg Loves(x, F(x)) \vee Loves(G(x), x)\}$$

KB as sentence:

$$(\forall x\ Animal(F(x)) \vee Loves(G(x), x)) \wedge (\forall x\ \neg Loves(x, F(x)) \vee Loves(G(x), x))$$

Move universal quantifiers outward (see "by hand" proof in last week's lecture):

$$\forall x\ (Animal(F(x)) \vee Loves(G(x), x)) \wedge (\neg Loves(x, F(x)) \vee Loves(G(x), x))$$

The resulting KB with implicit universal quantifiers is easier to work with:

$$\forall x\ \{Animal(F(x)) \vee Loves(G(x), x), \neg Loves(x, F(x)) \vee Loves(G(x), x)\}$$

Conversion to conjunctive normal form in first-order logic, example

Everyone who loves all animals is loved by someone

$$\forall x\ (\forall y\ \mathit{Animal}(y) \Rightarrow \mathit{Loves}(x, y)) \Rightarrow (\exists y\ \mathit{Loves}(y, x))$$

Conversion to conjunctive normal form in first-order logic, example

Everyone who loves all animals is loved by someone

$$\forall x\ (\forall y\ \mathit{Animal}(y) \Rightarrow \mathit{Loves}(x, y)) \Rightarrow (\exists y\ \mathit{Loves}(y, x))$$

Eliminate implications: $\forall x\ \neg(\forall y\ \mathit{Animal}(y) \Rightarrow \mathit{Loves}(x, y)) \vee (\exists y\ \mathit{Loves}(y, x))$

Conversion to conjunctive normal form in first-order logic, example

Everyone who loves all animals is loved by someone

$$\forall x\ (\forall y\ \mathit{Animal}(y) \Rightarrow \mathit{Loves}(x, y)) \Rightarrow (\exists y\ \mathit{Loves}(y, x))$$

Eliminate implications:

$$\forall x\ \neg(\forall y\ \mathit{Animal}(y) \Rightarrow \mathit{Loves}(x, y)) \vee (\exists y\ \mathit{Loves}(y, x))$$

$$\forall x\ \neg(\forall y\ \neg\mathit{Animal}(y) \vee \mathit{Loves}(x, y)) \vee (\exists y\ \mathit{Loves}(y, x))$$

Conversion to conjunctive normal form in first-order logic, example

Everyone who loves all animals is loved by someone

$$\forall x\ (\forall y\ \mathit{Animal}(y) \Rightarrow \mathit{Loves}(x,y)) \Rightarrow (\exists y\ \mathit{Loves}(y,x))$$

Eliminate implications:

$$\forall x\ \neg(\forall y\ \mathit{Animal}(y) \Rightarrow \mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

$$\forall x\ \neg(\forall y\ \neg\mathit{Animal}(y) \vee \mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

Move $\neg$ inwards:

$$\forall x\ (\exists y\ \neg(\neg\mathit{Animal}(y) \vee \mathit{Loves}(x,y))) \vee (\exists y\ \mathit{Loves}(y,x))$$

Conversion to conjunctive normal form in first-order logic, example

Everyone who loves all animals is loved by someone

$$\forall x\ (\forall y\ \mathit{Animal}(y) \Rightarrow \mathit{Loves}(x, y)) \Rightarrow (\exists y\ \mathit{Loves}(y, x))$$

Eliminate implications:

$$\forall x\ \neg(\forall y\ \mathit{Animal}(y) \Rightarrow \mathit{Loves}(x, y)) \vee (\exists y\ \mathit{Loves}(y, x))$$

$$\forall x\ \neg(\forall y\ \neg\mathit{Animal}(y) \vee \mathit{Loves}(x, y)) \vee (\exists y\ \mathit{Loves}(y, x))$$

Move $\neg$ inwards:

$$\forall x\ (\exists y\ \neg(\neg\mathit{Animal}(y) \vee \mathit{Loves}(x, y))) \vee (\exists y\ \mathit{Loves}(y, x))$$

$$\forall x\ (\exists y\ \neg\neg\mathit{Animal}(y) \wedge \neg\mathit{Loves}(x, y)) \vee (\exists y\ \mathit{Loves}(y, x))$$

Conversion to conjunctive normal form in first-order logic, example

Everyone who loves all animals is loved by someone

$$\forall x\ (\forall y\ \mathit{Animal}(y) \Rightarrow \mathit{Loves}(x,y)) \Rightarrow (\exists y\ \mathit{Loves}(y,x))$$

Eliminate implications:

$$\forall x\ \neg(\forall y\ \mathit{Animal}(y) \Rightarrow \mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

$$\forall x\ \neg(\forall y\ \neg\mathit{Animal}(y) \vee \mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

Move $\neg$ inwards:

$$\forall x\ (\exists y\ \neg(\neg\mathit{Animal}(y) \vee \mathit{Loves}(x,y))) \vee (\exists y\ \mathit{Loves}(y,x))$$

$$\forall x\ (\exists y\ \neg\neg\mathit{Animal}(y) \wedge \neg\mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

Eliminate double negation:

$$\forall x\ (\exists y\ \mathit{Animal}(y) \wedge \neg\mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

Conversion to conjunctive normal form in first-order logic, example

Everyone who loves all animals is loved by someone

$$\forall x\ (\forall y\ \mathit{Animal}(y) \Rightarrow \mathit{Loves}(x,y)) \Rightarrow (\exists y\ \mathit{Loves}(y,x))$$

Eliminate implications:

$$\forall x\ \neg(\forall y\ \mathit{Animal}(y) \Rightarrow \mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

$$\forall x\ \neg(\forall y\ \neg\mathit{Animal}(y) \vee \mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

Move $\neg$ inwards:

$$\forall x\ (\exists y\ \neg(\neg\mathit{Animal}(y) \vee \mathit{Loves}(x,y))) \vee (\exists y\ \mathit{Loves}(y,x))$$

$$\forall x\ (\exists y\ \neg\neg\mathit{Animal}(y) \wedge \neg\mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

Eliminate double negation:

$$\forall x\ (\exists y\ \mathit{Animal}(y) \wedge \neg\mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

Standardize variables:

$$\forall x\ (\exists y\ \mathit{Animal}(y) \wedge \neg\mathit{Loves}(x,y)) \vee (\exists z\ \mathit{Loves}(z,x))$$

Conversion to conjunctive normal form in first-order logic, example

Everyone who loves all animals is loved by someone

$$\forall x\ (\forall y\ \mathit{Animal}(y) \Rightarrow \mathit{Loves}(x,y)) \Rightarrow (\exists y\ \mathit{Loves}(y,x))$$

Eliminate implications:

$$\forall x\ \neg(\forall y\ \mathit{Animal}(y) \Rightarrow \mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

$$\forall x\ \neg(\forall y\ \neg\mathit{Animal}(y) \vee \mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

Move $\neg$ inwards:

$$\forall x\ (\exists y\ \neg(\neg\mathit{Animal}(y) \vee \mathit{Loves}(x,y))) \vee (\exists y\ \mathit{Loves}(y,x))$$

$$\forall x\ (\exists y\ \neg\neg\mathit{Animal}(y) \wedge \neg\mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

Eliminate double negation:

$$\forall x\ (\exists y\ \mathit{Animal}(y) \wedge \neg\mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

Standardize variables:

$$\forall x\ (\exists y\ \mathit{Animal}(y) \wedge \neg\mathit{Loves}(x,y)) \vee (\exists z\ \mathit{Loves}(z,x))$$

Skolemization:

$$\forall x\ (\mathit{Animal}(F(x)) \wedge \neg\mathit{Loves}(x,F(x))) \vee \mathit{Loves}(G(x),x)$$

Conversion to conjunctive normal form in first-order logic, example

Everyone who loves all animals is loved by someone

$$\forall x\ (\forall y\ \mathit{Animal}(y) \Rightarrow \mathit{Loves}(x,y)) \Rightarrow (\exists y\ \mathit{Loves}(y,x))$$

Eliminate implications:

$$\forall x\ \neg(\forall y\ \mathit{Animal}(y) \Rightarrow \mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

$$\forall x\ \neg(\forall y\ \neg\mathit{Animal}(y) \vee \mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

Move $\neg$ inwards:

$$\forall x\ (\exists y\ \neg(\neg\mathit{Animal}(y) \vee \mathit{Loves}(x,y))) \vee (\exists y\ \mathit{Loves}(y,x))$$

$$\forall x\ (\exists y\ \neg\neg\mathit{Animal}(y) \wedge \neg\mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

Eliminate double negation:

$$\forall x\ (\exists y\ \mathit{Animal}(y) \wedge \neg\mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

Standardize variables:

$$\forall x\ (\exists y\ \mathit{Animal}(y) \wedge \neg\mathit{Loves}(x,y)) \vee (\exists z\ \mathit{Loves}(z,x))$$

Skolemization:

$$\forall x\ (\mathit{Animal}(F(x)) \wedge \neg\mathit{Loves}(x,F(x))) \vee \mathit{Loves}(G(x),x)$$

Drop universal quantifiers:

$$(\mathit{Animal}(F(x)) \wedge \neg\mathit{Loves}(x,F(x))) \vee \mathit{Loves}(G(x),x)$$

Conversion to conjunctive normal form in first-order logic, example

Everyone who loves all animals is loved by someone

$$\forall x\ (\forall y\ \mathit{Animal}(y) \Rightarrow \mathit{Loves}(x,y)) \Rightarrow (\exists y\ \mathit{Loves}(y,x))$$

Eliminate implications:

$$\forall x\ \neg(\forall y\ \mathit{Animal}(y) \Rightarrow \mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

$$\forall x\ \neg(\forall y\ \neg\mathit{Animal}(y) \vee \mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

Move $\neg$ inwards:

$$\forall x\ (\exists y\ \neg(\neg\mathit{Animal}(y) \vee \mathit{Loves}(x,y))) \vee (\exists y\ \mathit{Loves}(y,x))$$

$$\forall x\ (\exists y\ \neg\neg\mathit{Animal}(y) \wedge \neg\mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

Eliminate double negation:

$$\forall x\ (\exists y\ \mathit{Animal}(y) \wedge \neg\mathit{Loves}(x,y)) \vee (\exists y\ \mathit{Loves}(y,x))$$

Standardize variables:

$$\forall x\ (\exists y\ \mathit{Animal}(y) \wedge \neg\mathit{Loves}(x,y)) \vee (\exists z\ \mathit{Loves}(z,x))$$

Skolemization:

$$\forall x\ (\mathit{Animal}(F(x)) \wedge \neg\mathit{Loves}(x,F(x))) \vee \mathit{Loves}(G(x),x)$$

Drop universal quantifiers:

$$(\mathit{Animal}(F(x)) \wedge \neg\mathit{Loves}(x,F(x))) \vee \mathit{Loves}(G(x),x)$$

Distribute $\vee$ over $\wedge$:

$$(\mathit{Animal}(F(x)) \vee \mathit{Loves}(G(x),x)) \wedge (\neg\mathit{Loves}(x,F(x)) \vee \mathit{Loves}(G(x),x))$$

Resolution inference rule in first-order logic

Resolution inference rule in first-order logic

For any first-order literals $a_1, a_2, \ldots, b_1, b_2, \ldots, c', c$:

$$\frac{a_1 \vee a_2 \vee \ldots \vee c', \qquad \neg c \vee b_1 \vee b_2 \vee \ldots}{\textsc{Subst}(\theta, a_1 \vee a_2 \vee \ldots \vee b_1 \vee b_2 \vee \ldots)}$$

where $\theta = \textsc{Unify}(c', c)$

Resolution inference rule in first-order logic

Resolution inference rule in first-order logic

For any first-order literals $a_1, a_2, \ldots, b_1, b_2, \ldots, c', c$:

$$\frac{a_1 \vee a_2 \vee \ldots \vee c', \qquad \neg c \vee b_1 \vee b_2 \vee \ldots}{\textsc{Subst}(\theta, a_1 \vee a_2 \vee \ldots \vee b_1 \vee b_2 \vee \ldots)}$$

where $\theta = \textsc{Unify}(c', c)$

For example, the first-order resolution inference rule applied to:

$KB = \{Animal(F(x)) \vee Loves(G(x), x), \neg Animal(F(Jack)) \vee Loves(G(Jack), Jack)\}$

with substitution $\theta = \{x/Jack\}$ derives:

$Loves(G(Jack), Jack) \vee Loves(G(Jack), Jack)$

(which is equivalent to $Loves(G(Jack), Jack)$)

First-order resolution inference algorithm

First-order resolution algorithm (basically the same as in propositional logic)

1. Add $\neg\alpha$ to KB, and convert all KB sentences to conjunctive normal form
2. Separate conjunctions into clauses, e.g. $\{(P(A) \vee P(B)) \wedge P(C)\}$ becomes $\{P(A) \vee P(B), P(C)\}$
3. Repeatedly apply the resolution inference rule, and return unsatisfiable ($KB \models \alpha$) if empty clause (contradiction) is derived, e.g. $\frac{P(A), \quad \neg P(A)}{}$, otherwise return satisfiable ($KB \not\models \alpha$)

First-order resolution inference algorithm example, problem

KB:

Everyone who loves all animals is loved by someone	$\forall x\ (\forall y\ \mathit{Animal}(y) \Rightarrow \mathit{Loves}(x, y)) \Rightarrow (\exists y\ \mathit{Loves}(y, x))$
Anyone who kills an animal is loved by no one	$\forall x\ (\exists z\ \mathit{Animal}(z) \land \mathit{Kills}(x, z)) \Rightarrow (\forall y\ \neg \mathit{Loves}(y, x))$
Jack loves all animals	$\forall x\ \mathit{Animal}(x) \Rightarrow \mathit{Loves}(\mathit{Jack}, x)$
Either Jack or Curiosity killed the cat who is named Tuna	$\mathit{Kills}(\mathit{Jack}, \mathit{Tuna}) \lor \mathit{Kills}(\mathit{Curiosity}, \mathit{Tuna})$ $\mathit{Cat}(\mathit{Tuna})$ $\forall x\ \mathit{Cat}(x) \Rightarrow \mathit{Animal}(x)$

Question:

Did Curiosity kill the cat?	$\mathit{Kills}(\mathit{Curiosity}, \mathit{Tuna})$

First-order resolution inference algorithm example, conjunctive normal form

$KB \cup \{\neg Kills(Curiosity, Tuna)\}$:

$\forall x\ (\forall y\ Animal(y) \Rightarrow Loves(x, y)) \Rightarrow (\exists y\ Loves(y, x))$	$Animal(F(x)) \vee Loves(G(x), x)$ $\neg Loves(x, F(x)) \vee Loves(G(x), x)$
$\forall x\ (\exists z\ Animal(z) \wedge Kills(x, z)) \Rightarrow (\forall y\ \neg Loves(y, x))$	$\neg Loves(y, x) \vee \neg Animal(z) \vee \neg Kills(x, z)$
$\forall x\ Animal(x) \Rightarrow Loves(Jack, x)$	$\neg Animal(x) \vee Loves(Jack, x)$
$Kills(Jack, Tuna) \vee Kills(Curiosity, Tuna)$	$Kills(Jack, Tuna) \vee Kills(Curiosity, Tuna)$
$Cat(Tuna)$	$Cat(Tuna)$
$\forall x\ Cat(x) \Rightarrow Animal(x)$	$\neg Cat(x) \vee Animal(x)$
$\neg Kills(Curiosity, Tuna)$	$\neg Kills(Curiosity, Tuna)$

Repeated applications of the first-order resolution inference rule

Initial $KB \cup \{\neg Kills(Curiosity, Tuna)\}$ in conjunctive normal form

$\neg Loves(y, x) \vee \neg Animal(z) \vee \neg Kills(x, z)$

$\neg Animal(x) \vee Loves(Jack, x)$

$Kills(Jack, Tuna) \vee Kills(Curiosity, Tuna)$

$Cat(Tuna)$

$\neg Loves(x, F(x)) \vee Loves(G(x), x)$

$\neg Cat(x) \vee Animal(x)$

$Animal(F(x)) \vee Loves(G(x), x)$

$\neg Kills(Curiosity, Tuna)$

Repeated applications of the first-order resolution inference rule

Initial $KB \cup \{\neg Kills(Curiosity, Tuna)\}$ in conjunctive normal form

$\neg Loves(y, x) \vee \neg Animal(z) \vee \neg Kills(x, z)$

$\neg Animal(x) \vee Loves(Jack, x)$

$Kills(Jack, Tuna) \vee Kills(Curiosity, Tuna)$

$Cat(Tuna)$

$\neg Loves(x, F(x)) \vee Loves(G(x), x)$

$\neg Cat(x) \vee Animal(x)$

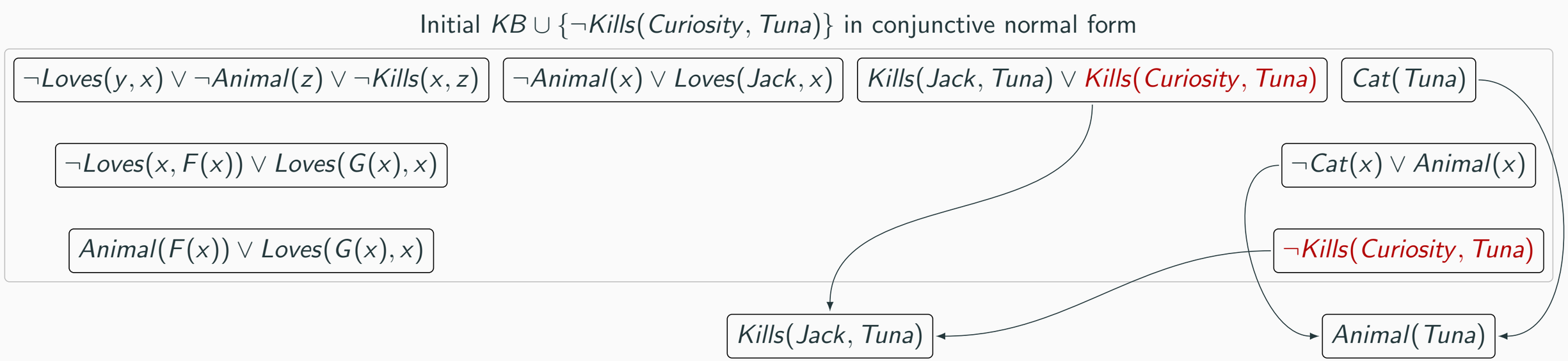
$Animal(F(x)) \vee Loves(G(x), x)$

$\neg Kills(Curiosity, Tuna)$

$Animal(Tuna)$

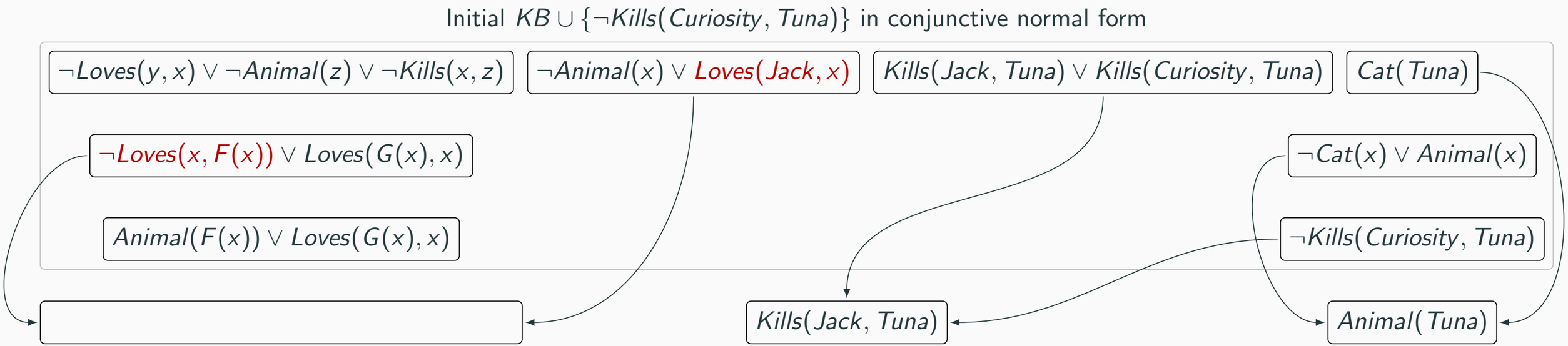
$\theta = \{x/Tuna\}$

Repeated applications of the first-order resolution inference rule



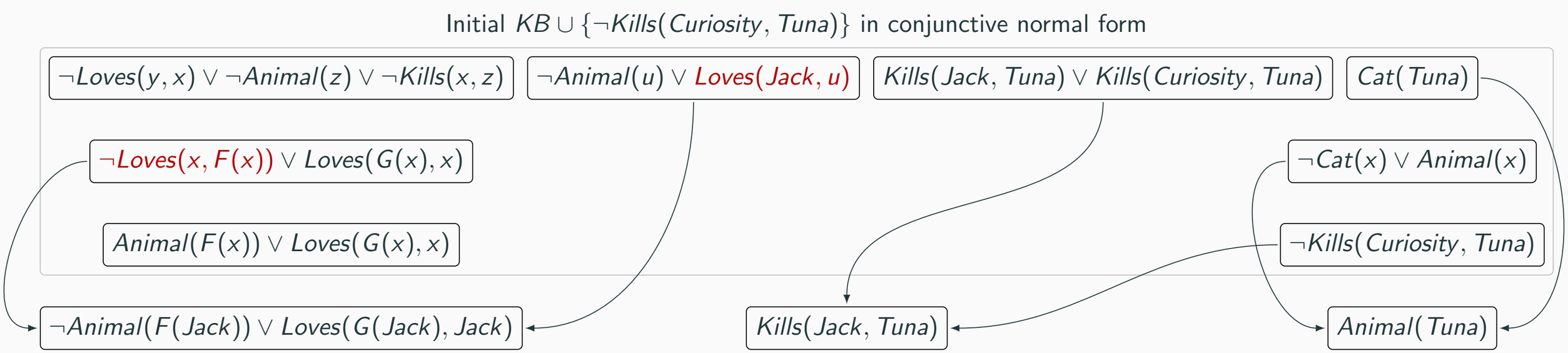
$\theta = \{\}$

Repeated applications of the first-order resolution inference rule



Note that $\forall x\ \neg Animal(x) \vee Loves(Jack, x) \equiv \forall u\ \neg Animal(u) \vee Loves(Jack, u)$

Repeated applications of the first-order resolution inference rule

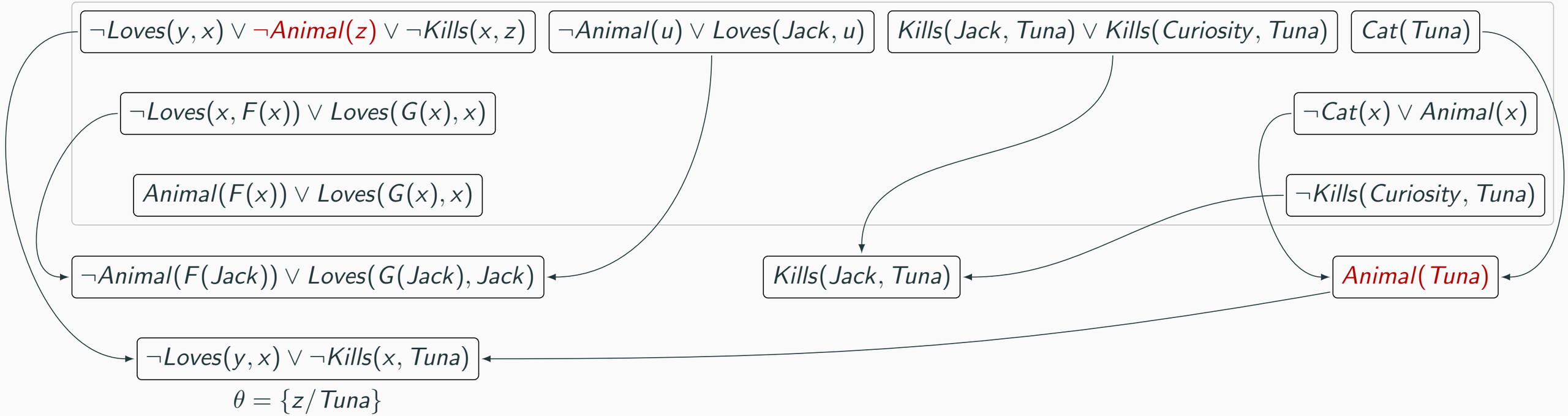


$\theta = \{x/Jack, u/F(Jack)\}$

Note that $\forall x\ \neg Animal(x) \vee Loves(Jack, x) \equiv \forall u\ \neg Animal(u) \vee Loves(Jack, u)$

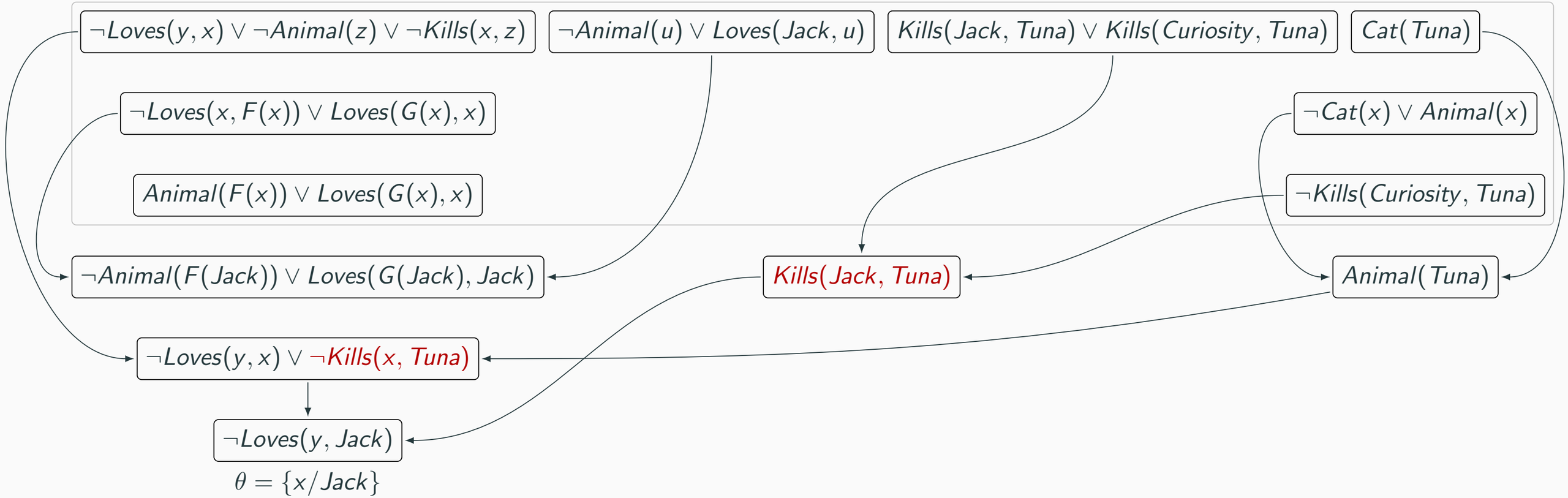
Repeated applications of the first-order resolution inference rule

Initial $KB \cup \{\neg Kills(Curiosity, Tuna)\}$ in conjunctive normal form



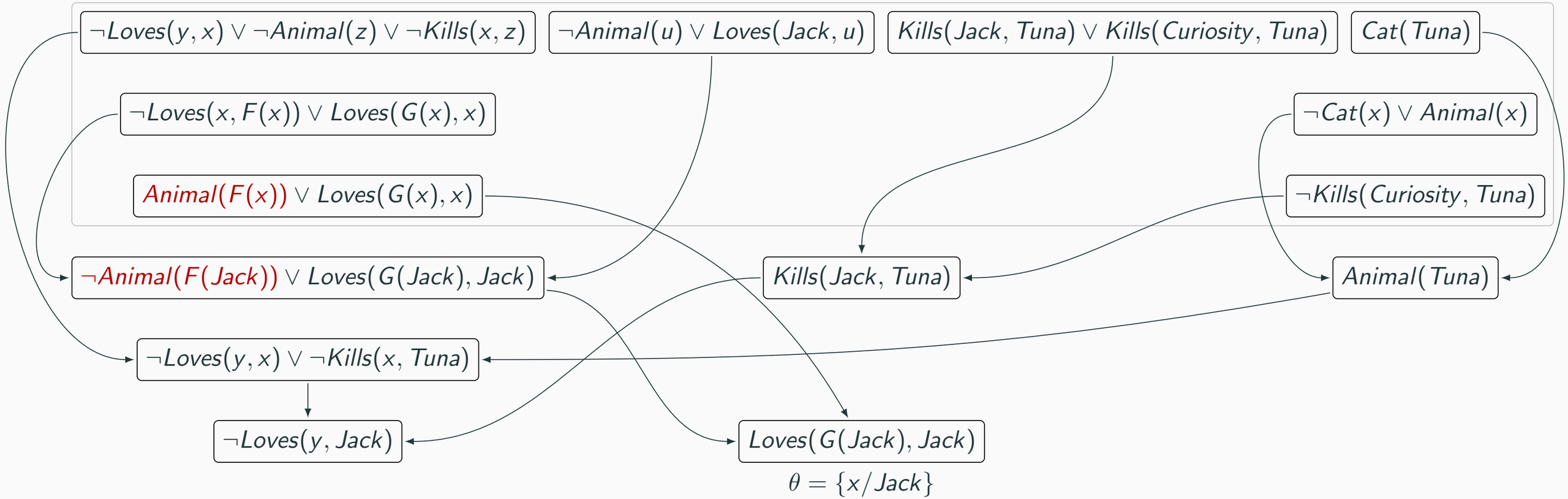
Repeated applications of the first-order resolution inference rule

Initial $KB \cup \{\neg Kills(Curiosity, Tuna)\}$ in conjunctive normal form



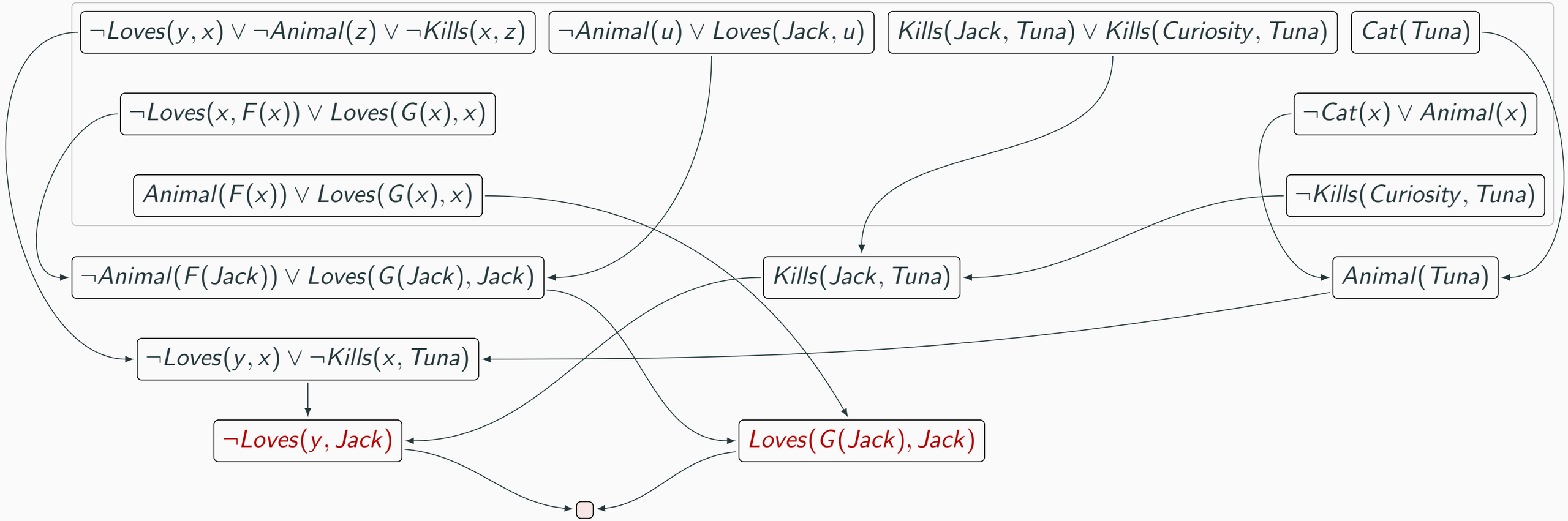
Repeated applications of the first-order resolution inference rule

Initial $KB \cup \{\neg Kills(Curiosity, Tuna)\}$ in conjunctive normal form



Repeated applications of the first-order resolution inference rule

Initial $KB \cup \{\neg Kills(Curiosity, Tuna)\}$ in conjunctive normal form



$\theta = \{y/G(Jack)\}$

$KB \models Kills(Curiosity, Tuna)$

Answer: yes, Curiosity killed the cat

Example strategies (heuristics)

- Unit preference: prefer shorter clauses

Example strategies (heuristics)

- Unit preference: prefer shorter clauses
- Set of support: prefer clauses with elements from a "handcrafted" set of clauses, e.g. the negated goal (question) for more goal oriented proofs

Example strategies (heuristics)

- Unit preference: prefer shorter clauses
- Set of support: prefer clauses with elements from a "handcrafted" set of clauses, e.g. the negated goal (question) for more goal oriented proofs
- Input resolution: prefer clauses from the original KB or goal

Example strategies (heuristics)

- Unit preference: prefer shorter clauses
- Set of support: prefer clauses with elements from a "handcrafted" set of clauses, e.g. the negated goal (question) for more goal oriented proofs
- Input resolution: prefer clauses from the original KB or goal
- Subsumption: keep the KB small by avoiding deriving clauses that are more specific than existing clauses in the KB, e.g. if the KB contains $\forall x\ P(x)$, there is no need to derive $P(A)$ or $P(A) \vee Q(B)$

Example strategies (heuristics)

- Unit preference: prefer shorter clauses
- Set of support: prefer clauses with elements from a "handcrafted" set of clauses, e.g. the negated goal (question) for more goal oriented proofs
- Input resolution: prefer clauses from the original KB or goal
- Subsumption: keep the KB small by avoiding deriving clauses that are more specific than existing clauses in the KB, e.g. if the KB contains $\forall x\ P(x)$, there is no need to derive $P(A)$ or $P(A) \vee Q(B)$
- Learning: prefer clauses based on the output of a machine learning model that has been trained on previous proofs
    - Inaccurate machine learning methods can still be useful for guiding search

Equality

Ways for inference algorithms to handle sentences with equality:

- Axiomatize equality: state that equality is reflexive, symmetric and transitive, and that we can substitute equals for equals in any predicate or function:
  $\forall x\ x = x$, $\forall x, y\ x = y \Rightarrow y = x$, $\forall x, y, z\ x = y \wedge y = z \Rightarrow x = z$,
  $\forall x, y\ x = y \Rightarrow (P_1(x) \Leftrightarrow P_1(y))$, $\ldots$,
  $\forall w, x, y, z\ w = y \wedge x = z \Rightarrow F_1(w, x) = F_1(y, z)$, $\ldots$
  such that e.g. the resolution inference algorithm can be used without modification

Equality

Ways for inference algorithms to handle sentences with equality:

- Axiomatize equality: state that equality is reflexive, symmetric and transitive, and that we can substitute equals for equals in any predicate or function:
  $\forall x\ x = x, \qquad \forall x, y\ x = y \Rightarrow y = x, \qquad \forall x, y, z\ x = y \wedge y = z \Rightarrow x = z,$
  $\forall x, y\ x = y \Rightarrow (P_1(x) \Leftrightarrow P_1(y)), \qquad \ldots,$
  $\forall w, x, y, z\ w = y \wedge x = z \Rightarrow F_1(w, x) = F_1(y, z), \qquad \ldots$
  such that e.g. the resolution inference algorithm can be used without modification
- Two additional inference rules, demodulation and paramodulation, which e.g. enables derivation of $\mathit{Birthdate}(\mathit{PaternalGrandfather}(\mathit{Bella}), 1926)$ from
  $\{\forall x\ \mathit{Father}(\mathit{Father}(x)) = \mathit{PaternalGrandfather}(x),$
  $\mathit{Birthdate}(\mathit{Father}(\mathit{Father}(\mathit{Bella})), 1926) \qquad \}$

Equality

Ways for inference algorithms to handle sentences with equality:

- Axiomatize equality: state that equality is reflexive, symmetric and transitive, and that we can substitute equals for equals in any predicate or function:
  $\forall x\ x = x$, $\forall x, y\ x = y \Rightarrow y = x$, $\forall x, y, z\ x = y \wedge y = z \Rightarrow x = z$,
  $\forall x, y\ x = y \Rightarrow (P_1(x) \Leftrightarrow P_1(y))$, $\ldots$,
  $\forall w, x, y, z\ w = y \wedge x = z \Rightarrow F_1(w, x) = F_1(y, z)$, $\ldots$
  such that e.g. the resolution inference algorithm can be used without modification
- Two additional inference rules, demodulation and paramodulation, which e.g. enables derivation of $\mathit{Birthdate}(\mathit{PaternalGrandfather}(\mathit{Bella}), 1926)$ from
  $\{\forall x\ \mathit{Father}(\mathit{Father}(x)) = \mathit{PaternalGrandfather}(x),$
  $\mathit{Birthdate}(\mathit{Father}(\mathit{Father}(\mathit{Bella})), 1926)\ \}$
- Equational unification: modifications to the unification algorithms to allow unifications of sentences such as $\mathit{Add}(1, 2)$ and $\mathit{Add}(2, 1)$, if $\forall x, y\ \mathit{Add}(x, y) = \mathit{Add}(y, x)$ is known